

Reporte de Avance

Plataforma MLOps para Pricing Intelligence

Instituto Tecnológico y de Estudios Superiores de Monterrey
MNA – Proyecto integrador (Gpo 10)
Team 46

Emanuel Flores Martinez – A01796497
Mario Javier Soriano Aguilera – A01384282
David Alberto Serrano Garcia – A01795935

24 de mayo de 2026

Estado general

MVP operativo

Function → Azure ML → Storage → SQL audit

Ambiente validado

staging

Sin datos unmasked ni producción

Evidencia SQL

1 corrida

Run auditado el 25-may-2026

Resumen ejecutivo

Durante esta etapa se construyó una primera base operativa de MLOps para Pricing Intelligence en Azure. El proyecto pasó de un diseño conceptual a un flujo ejecutable en la nube con infraestructura reproducible, datos masked, ejecución en Azure ML, outputs versionados en Storage y metadata consultable en Azure SQL.

El avance no representa todavía el producto final ni un modelo productivo de pricing. La contribución principal es haber validado la columna vertebral operativa: un disparador controlado, un compute ML separado del orquestador, persistencia de artefactos y una capa de auditoría metadata-only. Esto permite integrar posteriormente un modelo real sin rehacer la plataforma.

Integrantes

Integrante	Matricula
Emanuel Flores Martinez	A01796497
Mario Javier Soriano Aguilera	A01384282
David Alberto Serrano Garcia	A01795935

Objetivo del proyecto

El objetivo original es construir una capa MLOps que permita operar recomendaciones de pricing de forma reproducible, auditable y gobernada. La plataforma debe soportar validaciones de datos, ejecuciones de modelo, detección de drift, evidencia histórica y controles de seguridad.

En esta entrega el alcance se concentro en un MVP de bajo costo:

- Infraestructura Azure declarada con Bicep.
- Separación entre plataforma, código funcional y evidencia histórica.
- Orquestación con Azure Functions.
- Ejecución del flujo técnico en Azure Machine Learning.
- Outputs funcionales versionados en Storage/ADLS.
- Auditoría metadata-only en Azure SQL.
- Uso exclusivo de datos masked en `staging`.

Alcance detallado de la entrega

La entrega se puede leer como una plataforma mínima viable, no como un sistema de pricing terminado. La diferencia es importante: el valor de esta etapa está en probar que la infraestructura, la orquestación, la ejecución ML y la auditoría pueden trabajar juntas de forma repetible.

Dimension	Incluido en esta etapa	No incluido todavía
Datos	Dataset masked de muestra en <code>raw-masked</code> ; rechazo explícito de <code>raw-unmasked</code> en <code>staging</code> .	Ingesta desde sistemas fuente reales, masking productivo y reconciliación con datos históricos completos.
Modelo	Baseline operativo y scoring controlado para validar contratos.	Modelo campeón real, entrenamiento formal, registro de modelos y promoción entre ambientes.
MLOps	Orquestación, pipeline, outputs versionados y metadata SQL.	Rollback productivo, aprobaciones de negocio, alertas automáticas y tablero ejecutivo.
Infraestructura	Bicep para foundation y workload; <code>staging</code> operativo.	Producción, Hub-Spoke, Private Endpoints y Azure Data Factory (ADF).
Seguridad	Managed identities, RBAC, sin secretos en Git y sin account keys para Storage MLOps.	Autenticación enterprise del endpoint, red privada completa y políticas de retención aprobadas.

Organizacion de repositorios

El proyecto quedo separado para evitar mezclar responsabilidades de plataforma con codigo funcional de ciencia de datos.

Repositorio	Responsabilidad	Evidencia usada en el reporte
pricing-mlops-platform	IaC, Azure Functions, definicion de pipeline AML, scripts operativos, SQL audit y documentacion.	Bicep, <code>function_app.py</code> , YAML de Azure ML, SQL schema y runbooks.
pricing-mlops	Flujo funcional/data science: validacion, curated, scoring, drift, reportes y publicacion de artefactos.	Componentes <code>validate_prepare</code> , <code>score_evaluate</code> y <code>publish_outputs</code> .
pricing-mlops-eda	Referencia historica y analisis exploratorio.	Contexto original, no ruta operativa actual.

Arquitectura implementada

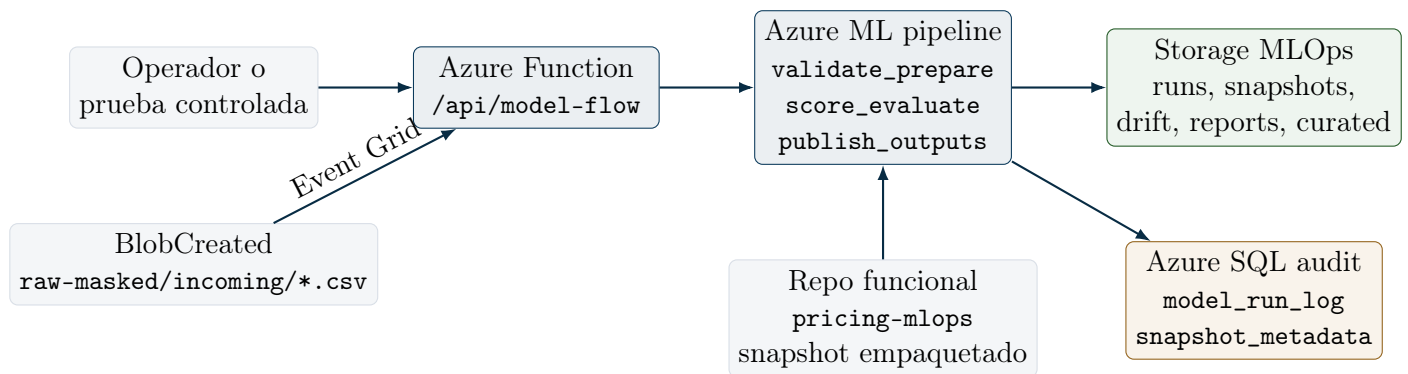


Figura 1: Flujo operativo actual del MVP.

La Function no ejecuta el modelo. Su responsabilidad es validar parametros, rechazar entradas inseguras y someter el job de Azure ML. Azure ML ejecuta el flujo funcional del repositorio `pricing-mlops`. Storage conserva los artefactos completos, mientras que SQL guarda metadata consultable para auditoria.

Flujo tecnico paso a paso

1. Un operador llama la Function HTTP o llega un evento de BlobCreated desde `raw-masked/incoming/*.csv`.
2. La Function valida ambiente, owner, container, extension del archivo y path relativo seguro.
3. La Function genera un `run_id`, calcula el prefijo esperado de salida y somete un pipeline job a Azure ML.

4. Azure ML ejecuta `validate_prepare`: descarga el CSV masked, valida el contrato inicial y crea curated intermedio.
5. Azure ML ejecuta `score_evaluate`: aplica scoring controlado, calcula drift basico y produce artefactos locales del run.
6. Azure ML ejecuta `publish_outputs`: publica los seis artefactos funcionales en Storage y, si esta habilitado, escribe metadata en SQL.
7. El operador puede consultar evidencia por tres vias: Azure ML Jobs, Storage Blob y Azure SQL audit.

Componente	Responsabilidad actual	Criterio de exito
<code>model-flow HTTP</code>	Entrada manual/controlada para iniciar una corrida.	Responde 202 con <code>run_id</code> y nombre del job AML.
Event Grid trigger	Entrada automatica por llegada de CSV masked.	Acepta solo <code>raw-masked/incoming/*.csv</code> .
<code>validate_prepare</code>	Validacion inicial y generacion de curated.	Produce estado preparado sin tocar datos unmasked.
<code>score_evaluate</code>	Scoring baseline y drift basico.	Genera snapshot, drift log y reporte.
<code>publish_outputs</code>	Publicacion en Storage y SQL metadata.	Seis artefactos en Blob y registros SQL cuando el sink esta activo.

Integracion actual con Azure ML Pipeline

La integracion actual con Azure ML ya representa un avance importante frente a ejecutar todo en una Function o en un script local. El pipeline aparece en Azure ML Studio con tres nodos visibles: `validate_prepare`, `score_evaluate` y `publish_outputs`. Esto permite separar responsabilidades, observar cada etapa y mantener a Azure ML como compute principal del flujo.

Sin embargo, la version actual todavia no es un DAG de datos completo. Los nodos se ejecutan en secuencia mediante `depends_on`, pero una parte del estado intermedio se pasa por Blob Storage bajo `artifacts/component-state/<run_id>/`. Esto es robusto e idempotente para el MVP, pero Azure ML no interpreta esos blobs como dependencias nativas de datos entre componentes.

Aspecto	Estado actual	Limitacion
Definicion del pipeline	YAML <code>pricing-mlops-pipeline.yml</code> .	Menos expresivo para contratos de datos complejos que SDK v2 DSL.
Secuencia de nodos	<code>depends_on</code> fuerza orden de ejecucion.	El orden existe, pero no modela datos como edges nativos.
Estado intermedio	Blobs bajo <code>component-state/<run_id></code> .	Azure ML Studio no puede inferir plenamente lineage de inputs/outputs.

Aspecto	Estado actual	Limitacion
Outputs finales	Se publican al layout funcional de Storage MLOps.	Correcto para auditoria, pero separado de outputs nativos de componentes AML.
Datastore	Uso operativo del storage funcional por identidad.	Falta formalizar un datastore AML explicito para dependencias intermedias.

Oportunidad: Pipeline Azure ML como DAG real

La oportunidad de mejora es evolucionar el pipeline a un DAG real de datos con Azure ML SDK v2 DSL, componentes registrados y contratos explicitos de entrada/salida. En vez de coordinar el estado intermedio por convencion de blobs, cada componente expondría outputs nativos tipo `uri_folder` que el siguiente componente consumiría directamente.

```
validate_prepare.outputs.prepared_data
-> score_evaluate.inputs.prepared_data
```

```
score_evaluate.outputs.run_artifacts
-> publish_outputs.inputs.run_artifacts
```

La mejora propuesta conserva el layout final de evidencia en Storage, pero hace que Azure ML entienda mejor el grafo, el lineage y las dependencias. Esto tambien reduce ambigüedad al diagnosticar fallas: si `score_evaluate` falla, Azure ML sabría con mayor claridad que input de datos recibio desde `validate_prepare`.

Cambio propuesto	Beneficio	Validacion necesaria
Implementar version SDK v2 DSL en <code>mlops/azureml/</code> .	Pipeline mas declarativo, componible y legible.	Mantener YAML actual como fallback hasta validar estabilidad.
Registrar componentes con interfaces explicitas.	Contratos claros para <code>prepared_data</code> y <code>run_artifacts</code> .	Confirmar que Studio muestre dependencias de datos reales.
Usar <code>inputs/outputs uri_folder</code> .	Lineage nativo entre componentes.	Verificar rutas generadas y persistencia esperada.
Crear datastore AML explicito al Storage funcional.	Evita depender del artifact store implicito.	Validar RBAC sin account keys ni connection strings.
Ejecutar con managed identity.	Mantiene el modelo de seguridad actual.	Confirmar permisos de lectura/escritura por componente.

Servicios desplegados

Servicio	Recurso	Rol
Resource Group	rg-pricing-mlops-staging	Ambiente operativo del MVP.
Storage MLOps	stpmlopsvwtrhrocbtxq2	Data lake funcional para inputs masked y outputs.
Azure ML Workspace	mlw-pricing-mlops-stg-v2-vwtrhr	Ejecucion de pipeline serverless.
Azure Function App	func-pricing-mlops-staging-vwtrhr	Orquestador HTTP y Event Grid.
Azure SQL Database	pricing_mlops_audit	Auditoria metadata-only.
Storage runtime AML	stamlpmlpsstgvmwtrhr	Logs, snapshots y artefactos internos de Azure ML.
Storage Function host	stfnvwtrhrocbtxq2	Estado tecnico de Azure Functions.
Managed Identity	id-pricing-mlops-aml-staging	Acceso sin secretos desde jobs AML.

Capas de almacenamiento y auditoria

La separacion de storages reduce ambigüedad operativa. El Storage MLOps principal contiene evidencia funcional del flujo. El Storage runtime de Azure ML guarda artefactos internos del servicio, como logs, snapshots y environments. El Storage de Function es un requisito tecnico del runtime de Azure Functions y no debe interpretarse como data lake.

Capa	Ejemplo	Interpretacion
Input masked	raw-masked/samples/sample_pricing.csv	Figura controlada usada para validar el flujo.
Curated	curated/... curated_pricing.csv	Datos procesados listos para consumo posterior.
Snapshot	snapshots/... model_output_snapshot.csv	Resultado de scoring publicado por corrida.
Drift log	drift-logs/... model_drift_log.json	Resultado del semaforo y metricas de desviacion.
Run log	runs/...model_run_log.json	Manifest de ejecucion y metadata funcional.
SQL audit	dbo.model_run_log	Indice consultable para auditoria y reportes.

Avance contra el plan original

Tema	Plan original	Estado actual	Estado
Orquestacion	Azure Functions y Azure Data Factory (ADF) para coordinar ingesta y flujo.	Azure Functions ya opera HTTP y Event Grid; ADF se pospuso.	Parcial
Compute ML	Azure ML para validacion, entrenamiento y scoring.	Azure ML ejecuta pipeline de 3 componentes.	MVP
Storage/ADLS	Capas raw, curated y snapshots.	Containers funcionales para raw-masked, curated, runs, snapshots, drift-logs, reports y artifacts.	MVP
Azure SQL	Espina dorsal de auditoria.	Tablas <code>model_run_log</code> , <code>model_output_snapshot_metadata</code> y <code>data_quality_log</code> .	MVP
Drift	PSI, KS y semaforos por impacto de negocio.	Drift basico; semaforo registrado en la corrida auditada.	Parcial
Calidad de datos	Great Expectations y reglas criticas.	Validacion inicial de contrato; reglas de negocio avanzadas pendientes.	Parcial
Modelo real	Modelo campeon versionado con rollback.	Baseline/scoring controlado; modelo real pendiente.	Pendiente
Seguridad avanzada	Hub-Spoke, Private Endpoints, Key Vault y red cerrada.	RBAC, identities y no secretos; Function key temporal y red privada pendiente.	Parcial
Produccion	Ambiente productivo gobernado.	No existe prod en IaC ni parameters.	Fuera alcance

Drift respecto al plan original

En este reporte se usa la palabra drift en dos sentidos distintos. El primero es el **drift del proyecto**, es decir, los cambios de alcance y orden de implementacion respecto al diseno original. El segundo es el **drift de datos/modelo**, que es la desviacion estadistica que la plataforma debe detectar durante las corridas de pricing.

El drift del proyecto fue principalmente de alcance y orden de implementacion, no de objetivo. La meta de una plataforma MLOps auditable para pricing se mantiene.

- **ADF se pospuso.** Para el MVP, Event Grid + Azure Functions cubre el disparo automatico sobre archivos `raw-masked/incoming/*.csv`. ADF sigue teniendo sentido si aparecen multiples fuentes, calendarios y transformaciones previas.
- **Private Endpoints y Hub-Spoke se pospusieron.** La razon principal fue costo, complejidad y tiempo academico. El MVP conserva controles base: sin secretos en Git, identities, RBAC y datos masked.
- **GitHub Actions no opera ML.** Se limito a CI/CD y validaciones. La operacion ML vive en Function + Azure ML.

- **Container Apps no era parte del plan original.** Fue una exploracion tecnica intermedia para probar ejecucion remota, pero se retiro porque el diseno original y la ruta actual son Azure Functions como orquestador y Azure ML como compute ML.
- **SQL se incorporo antes de cerrar el modelo real.** Esto mejora auditoria, pero aun requiere endurecimiento y correccion de URIs finales.

Decision	Motivo	Impacto positivo	Deuda generada
Posponer ADF	Reducir complejidad inicial.	MVP mas rapido con Function + Event Grid.	Falta ingesta multi-fuente calendarizada.
Posponer red privada	Costo y tiempo academico.	Menor friccion de despliegue.	Falta hardening enterprise.
Descartar Container Apps	No pertenece al diseno objetivo y duplicaba responsabilidades de AML.	Menos superficie operativa y mejor alineacion con el plan original.	Mantener solo evidencia historica del PoC.
Usar baseline	Modelo real no cerrado.	Permite probar plataforma completa.	Falta calidad predictiva real.
Separar Storage runtime AML	Evitar mezclar artifacts internos con outputs funcionales.	Mejor gobierno de artefactos.	Requiere limpieza legacy controlada.
Agregar SQL audit	Mejorar trazabilidad desde temprano.	Consultas operativas sin leer blobs.	Falta guardar URIs finales de Blob.

Drift de datos y modelo

El drift funcional actual ya genera un semaforo y lo publica en Storage y SQL, pero todavia es una version basica. Su valor en esta etapa es probar el contrato operativo: el pipeline calcula un resultado, lo registra como `drift_status` y deja evidencia por run. La siguiente iteracion debe convertir ese semaforo en una decision de negocio calibrada.

Senal	Metodo	objetivo	Umbral original	Interpretacion
rs1priceusd quantity	y	PSI	0.10 amarillo / 0.25 rojo	Cambios fuertes en rangos de precio o volumen de venta.
elasticity_mean		KS	0.05 amarillo / 0.15 rojo	Cambio en el comportamiento elastico del mercado.
P85 - P20		KS	0.10 amarillo / 0.20 rojo	Compresion del rango de precios sugeridos.
Tasa P20_Was_Adjusted		Z-test de proporciones	$z > 2.0$ amarillo / $z > 3.5$ rojo	Aumento de casos que tocan el piso de margen.

Para evitar falsas alarmas, el semaforo no debe depender solo de una metrica global. La mejora recomendada es calcular drift por columnas criticas y por segmentos de negocio, guardar el detalle en `model_drift_log.json`, registrar el resumen en SQL y conservar la justificacion del color final. Asi el reporte no solo dira `red`, `yellow` o `green`; tambien dira que senal causo la alerta, con que umbral y sobre que poblacion.

Evidencia de ejecucion

La evidencia se consulto en Azure SQL y Storage el 24 de mayo de 2026, hora local America/Tijuana. Algunos identificadores tecnicos, como `run_id`, usan fecha UTC.

Campo	Valor
Run ID	20260525T021122Z-function
Ambiente	staging
Estado	succeeded
Trigger	manual
Filas procesadas	3
Drift status	red
Modelo / ref	PoC/model-flow-template
Commit	f4efe0245695891c400072eb2e1d341145129e42
Inicio UTC	2026-05-25T02:16:31.175029+00:00
Fin UTC	2026-05-25T02:16:31.175520+00:00

Tabla 11: Ultima corrida registrada en `dbo.model_run_log`.

Tabla SQL	Registros	Uso
<code>model_run_log</code>	1	Metadata principal de corrida.
<code>model_output_snapshot_metadata</code>	1	Metadata de snapshot publicado.
<code>data_quality_log</code>	0	Reservada para checks detallados.

Tabla 12: Conteos actuales en la base `pricing_mlops_audit`.

Tabla	Campos relevantes	Uso academico/operativo
<code>model_run_log</code>	<code>run_id</code> , <code>status</code> , <code>row_count</code> , <code>drift_status</code> , <code>model_commit_sha</code> .	Demuestra trazabilidad por ejecucion, ambiente, input y version de codigo/modelo.
<code>model_output_snapshot_metadata</code>	<code>snapshot_uri</code> , <code>row_count</code> , <code>output_schema_version</code> .	Permite ubicar el resultado publicado y validar version de contrato de salida.
<code>data_quality_log</code>	<code>check_name</code> , <code>check_status</code> , <code>observed_value</code> , <code>threshold_value</code> .	Debe convertirse en evidencia de compuertas de calidad por run.

Container	Artefacto verificado en Storage	Bytes
runs	model_run_log.json	1122
snapshots	model_output_snapshot.csv	610
drift-logs	model_drift_log.json	648
reports	report.md	346
artifacts	curated_pricing.csv	351
curated	curated_pricing.csv	351

Tabla 14: Outputs funcionales versionados en Storage para el run auditado.

Lectura de resultados

La corrida registrada es exitosa desde el punto de vista de plataforma: el pipeline termino, publico outputs y escribio metadata SQL. El semaforo **red** no significa que la plataforma fallo; significa que el motor de drift clasifico el dataset de muestra como fuera de tolerancia bajo la logica actual. Para una entrega academica, esto es util porque demuestra que el campo de drift ya viaja desde el flujo funcional hasta SQL y Storage.

La interpretacion correcta es:

- **Exito tecnico:** Function, Azure ML, Storage y SQL se integraron de punta a punta.
- **Limitacion analitica:** el drift aun no usa todas las metricas propuestas en el plan original.
- **Limitacion de auditoria:** SQL tiene registros, pero los campos de URI deben apuntar a Blob Storage final, no a rutas temporales locales.
- **Limitacion de escala:** la evidencia usa 3 filas de muestra; falta validar con datasets historicos masked de mayor volumen.

Resultados ya logrados

- IaC organizada en foundation, workloads y parameter files por ambiente.
- Ambiente **staging** activo con Storage, Azure ML, Function, Event Grid y SQL audit.
- Pipeline Azure ML visible con tres nodos: **validate_prepare**, **score_evaluate** y **publish_outputs**.
- Flujo remoto ejecutado sin usar GitHub Actions como compute operativo.
- Seis outputs funcionales publicados en Storage para una corrida auditada.
- Base SQL con metadata de corrida y snapshot.
- Separacion de Storage funcional MLOps, Storage runtime Azure ML y Storage tecnico de Function.
- Politica practica de datos: **staging** no usa **raw-unmasked**.

Entregable	Que demuestra	Archivo o recurso
Infraestructura declarativa	El ambiente se puede reconstruir y revisar por código.	<code>infra/foundation</code> , <code>infra/workloads</code> .
Function orchestrator	El orquestador valida entradas y somete jobs AML.	<code>mlops/functions/function_app.py</code> .
Pipeline AML	El compute ML corre fuera de la Function.	<code>mlops/azureml/pricing-mlops-pipeline.yml</code> .
SQL audit schema	Hay una base para auditoria consultable.	<code>mlops/sql/001_create_audit_tables.sql</code> .
Runbook operativo	El flujo se puede ejecutar y diagnosticar.	<code>docs/operations.md</code> , <code>docs/sql-audit-runbook.md</code> .
Reporte academico	El avance queda documentado con evidencia.	Este documento LaTeX/PDF.

Hallazgos y oportunidades de mejora

Oportunidad	Situación actual	Mejora recomendada
Auditoria SQL de URIs	SQL registra la corrida, pero <code>snapshot_uri</code> y <code>artifact_manifest_uri</code> aparecen como rutas temporales locales <code>/tmp/...</code> .	Guardar las rutas Blob finales para que SQL sea evidencia auditable directa.
Drift estadístico	La corrida registra <code>red</code> , pero el motor aun es básico.	Implementar PSI, KS y Z-test por métricas de negocio, con umbrales aprobados.
Calidad de datos	<code>data_quality_log</code> existe pero no tiene registros.	Persistir checks por run: nullos, unicidad, monotonicidad y margen mínimo.
Pipeline AML como DAG real	Hay tres nodos visibles con <code>depends_on</code> ; parte del estado intermedio vive en Blob y no como edge nativo de datos.	Evolucionar a SDK v2 DSL con componentes registrados, datastore explícito e <code>inputs/outputs uri_folder</code> .
Modelo real	El flujo usa <code>baseline/scoring</code> controlado.	Integrar modelo real como paquete, componente AML o adaptador estable.
Seguridad endpoint	Function key funciona como control temporal.	Migrar a Entra ID/Easy Auth o API Management.
Retención	Existen outputs funcionales y artifacts runtime.	Definir reglas de lifecycle por clase de datos y evidencia académica.

Oportunidad	Situacion actual	Mejora recomendada
Ambiente validation	Esta preparado, pero no opera como paso formal.	Usarlo como antesala para pruebas controladas antes de cualquier ambiente productivo.

Riesgos principales

Riesgo	Efecto si no se atiende	Mitigacion recomendada
Modelo real no integrado	El proyecto se queda como plataforma sin validacion predictiva.	Definir contrato del modelo y conectar un adaptador independiente del orquestador.
Drift basico	El semaforo podria no representar riesgo de negocio real o podria generar falsas alarmas.	Calibrar PSI, KS y Z-test con historicos, guardar detalle por metrica y pedir aprobacion de stakeholders.
Auditoria incompleta de URIs	SQL no serviria como indice directo a evidencia final.	Persistir rutas <code>https://...blob.core.windows.net/...</code> o <code>azureml://...</code> finales.
Endpoint con Function key	Control suficiente para MVP, debil para operacion enterprise.	Migrar a Entra ID, Easy Auth o API Management.
Artifacts legacy	Costos y confusion entre Storage funcional y runtime.	Aplicar inventario y retencion por clase de artefacto.
Pipeline sin dependencias de datos nativas	Azure ML puede ejecutar en orden, pero no captura completamente lineage, contratos intermedios ni diagnostico de datos entre nodos.	Migrar a DAG real con outputs <code>prepared_data</code> y <code>run_artifacts</code> conectados como inputs nativos.

Siguientes pasos

Prioridad	Paso	Resultado esperado	Horizonte
Alta	Corregir URIs SQL de snapshot y manifest.	Auditoria SQL apunta directo a evidencia final en Blob.	Corto plazo
Alta	Persistir checks en <code>data_quality_log</code> .	Compuertas de calidad visibles por run.	Corto plazo
Alta	Definir contrato del modelo real.	Entradas, salidas y versionado listos para integracion.	Corto plazo
Media	Implementar PSI, KS y Z-test con detalle por metrica.	Drift alineado con el plan original y explicable por run.	Mediano plazo

Prioridad	Paso	Resultado esperado	Horizonte
Media	Crear adaptador del modelo.	Modelo integrable sin cambiar Function ni pipeline base.	Mediano plazo
Media	Migrar pipeline a SDK v2 DSL.	Dependencias de datos nativas, lineage claro y DAG real en Azure ML.	Mediano plazo
Media	Preparar ambiente validation .	Pruebas controladas antes de promocion.	Mediano plazo
Baja	Migrar endpoint a Entra ID/API Management.	Autenticacion enterprise.	Mediano plazo
Baja	Definir lifecycle de artifacts.	Menor costo y limpieza gobernada.	Mediano plazo
Baja	Crear dashboard operativo.	Lectura ejecutiva de corridas, drift y estado.	Largo plazo

Criterios de cierre para la siguiente entrega

Para que la siguiente entrega muestre madurez adicional, se recomienda cerrarla con evidencia concreta de:

- Al menos una corrida con URIs finales de Blob registradas correctamente en SQL.
- Registros en `data_quality_log` para checks de contrato, nullos, unicidad y reglas de negocio.
- Drift calculado con al menos una metrica estadistica del plan original.
- Ejecucion en **validation** o justificacion documentada para mantener solo **staging**.
- Contrato firmado del modelo real o baseline formal aprobado.

Anexo visual: referencia del diseno original

El diseno original planteo una arquitectura enterprise con red privada, Azure Data Factory (ADF), Azure Functions, Azure ML, SQL y Key Vault. Las siguientes imagenes se conservan como referencia historica del plan inicial.

Conclusion

El proyecto ya cuenta con una base MLOps funcional para Pricing Intelligence. La plataforma puede recibir una solicitud, ejecutar un pipeline en Azure ML, publicar evidencia versionada y registrar metadata en SQL. El avance mas relevante es que la arquitectura ya no es solo un diagrama: existe una ejecucion verificable.

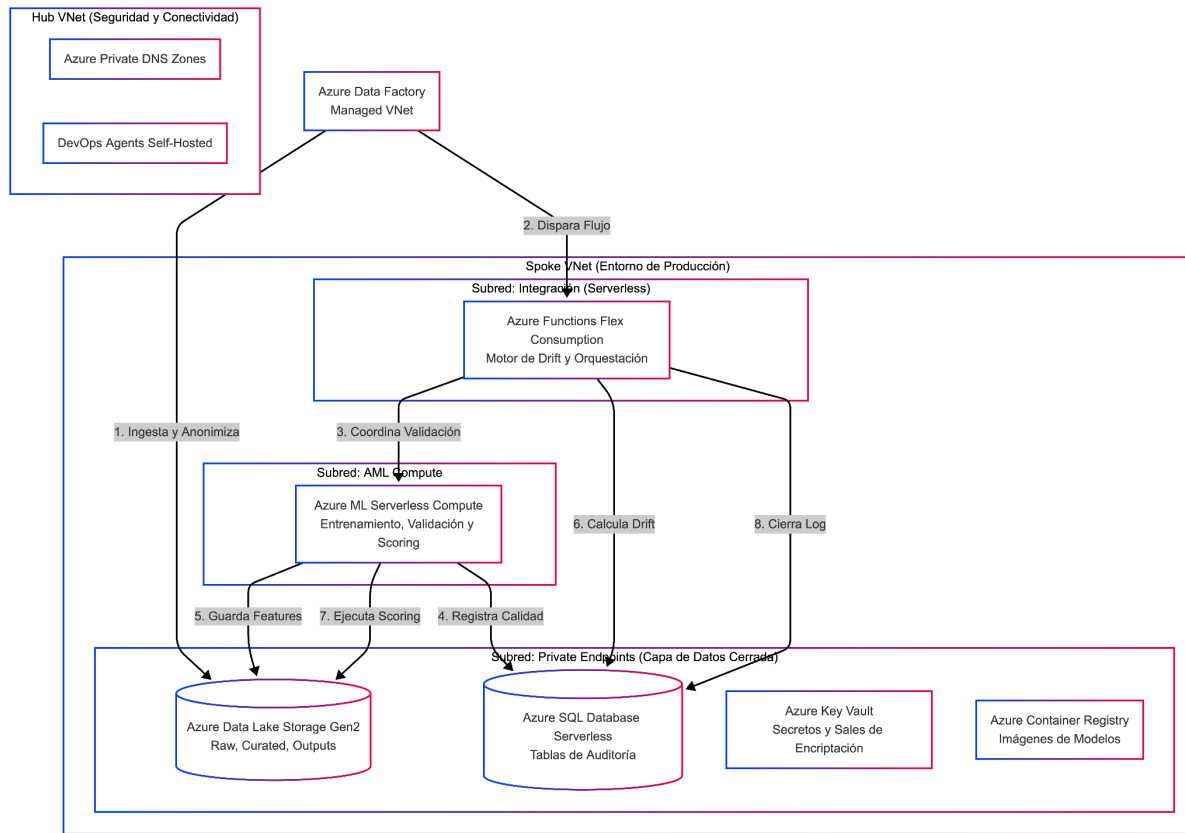


Figura 2: Topología objetivo del diseño técnico original.

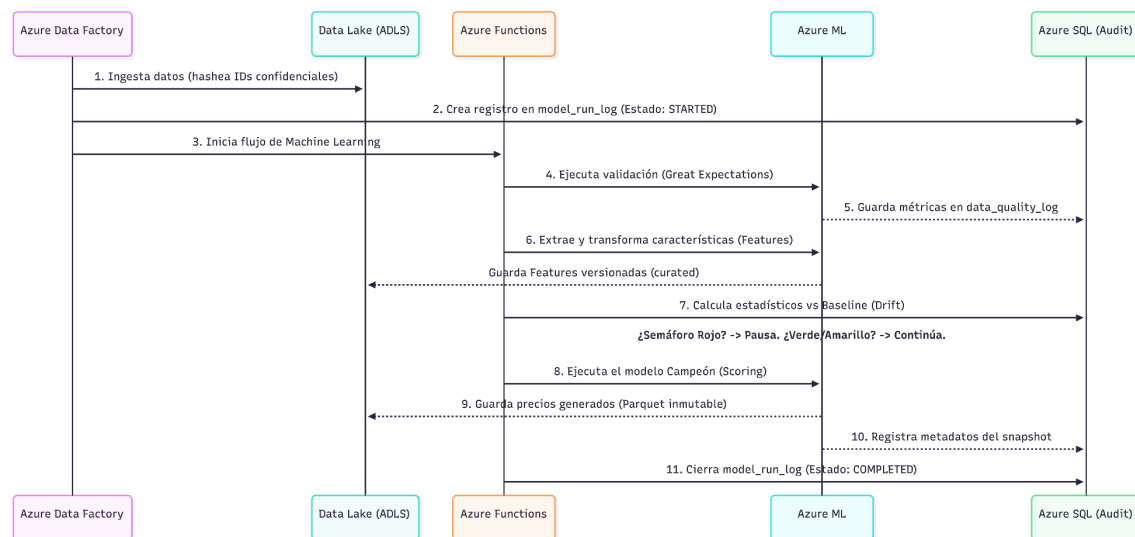


Figura 3: Sequence flow objetivo del diseño técnico original.

El principal trabajo pendiente es convertir el MVP en una plataforma mas cercana al diseño enterprise original: modelo real, drift estadístico robusto, reglas de calidad persistidas, autenticación fortalecida, pipeline con dependencias de datos nativas y auditoría SQL con URIs finales de Storage.